MODULE 2

Threat Modeling & Risk Assessment

Vlad Skorokhod

© 2026 Vlad Skorokhod

SECTION 1

Threat Modeling for AI Systems

- Identify threats early (cheap fixes) –
- Protect critical assets –
- Avoid costly post-deployment fixes –
- Build security into architecture

Why Threat Modeling Matters

Security failures are usually design failures

Attackers don't need bugs: just **design gaps**

Fixing issues:

- **Design:** minutes
- **Production:** months + $$$

AI systems:

- Larger attack surface
- New failure modes (data, models, prompts)

Insurance impact:

Fraud evasion

Data leakage

Model manipulation

© 2026 Vlad Skorokhod

Identify potential attacks before they happen, not after a breach

Prevent Attacks Proactively

Identify threats at design time, when fixes are cheap.

Protect Critical Assets

Know what matters: data, credentials, IP, availability.

Avoid Costly Late Fixes

Security added after deployment means patches, incidents, and breaches.

Secure by Design

Build security into architecture choices, not bolted on afterwards.

Analogy: planning home security:

Identify valuables → Map the house → Think like a burglar → Evaluate risks → Add protections

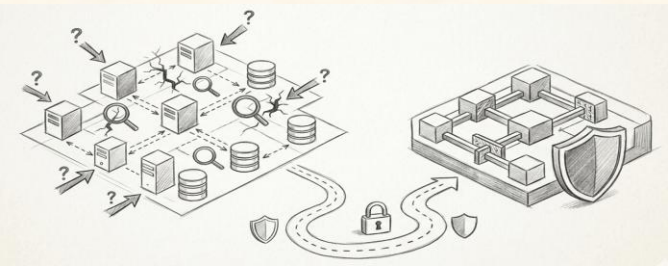
© 2026 Vlad Skorokhod

What Is Threat Modeling?

Structured way to ask: "What can go wrong?"

Core idea:

- Think like an attacker
  - before writing code



Output:

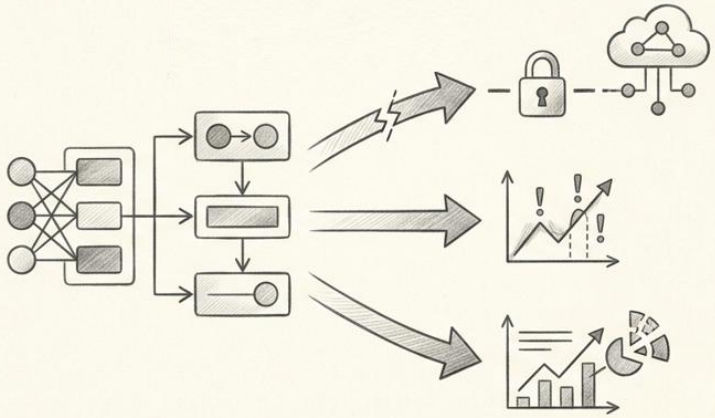
- Threat list
- Prioritized risks
- Mitigation

© 2026 Vlad Skorokhod

The 5-Step Process

1. **Define system** *(DFD + boundaries)*
2. **Identify assets** *(what matters)*
3. **Identify threats** *(STRIDE + AI Specific Framew*
4. **Assess risk** *(likelihood × impact)*
5. **Mitigate** *(controls)*

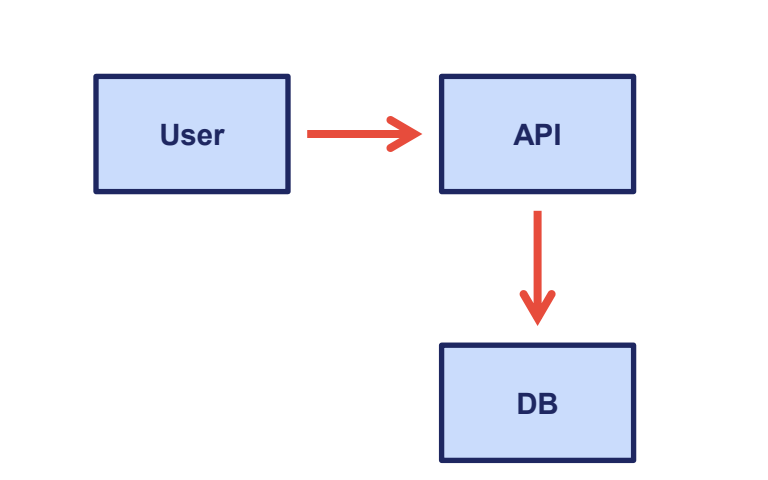
Deliverable = Threat register



Step 1: Define the System

Draw:

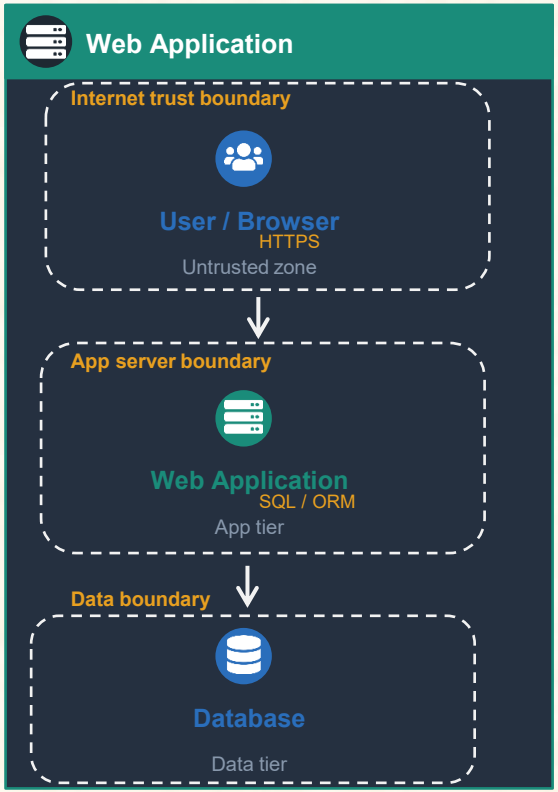
- Components
- Data flows
- Trust boundaries



Each arrow = attack path

© 2026 Vlad Skorokhod

Define the System: Example



Actors:
Who uses the system? Users, admins, external APIs, automated processes. Each actor = a potential attacker.

Components:
Web servers, APIs, databases, message queues, CDNs, third-party services. List every process that handles your data.

Data Flows:
Every arrow between components. Label with: protocol (HTTPS/SQL), direction, and what data is sent.

Trust Boundaries:
Dashed line between zones of different trust. Anything crossing a boundary needs validation + authentication.

Data Stores:
Databases, caches, file systems, logs. What is stored? Who can access? How is it protected at rest?

Data crossing a boundary must be validated. Example: internet → web server; web server → database.

© 2026 Vlad Skorokhod

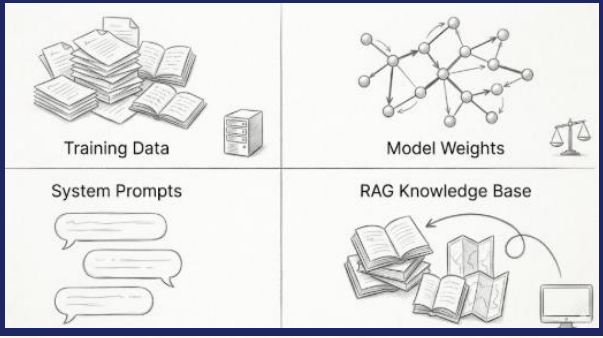
Step 2: Identify Assets

Examples:

Traditional systems

- Credentials
- Databases
- APIs

AI systems



If you can't name it → you can't protect it

© 2026 Vlad Skorokhod

Identify Assets Example 1

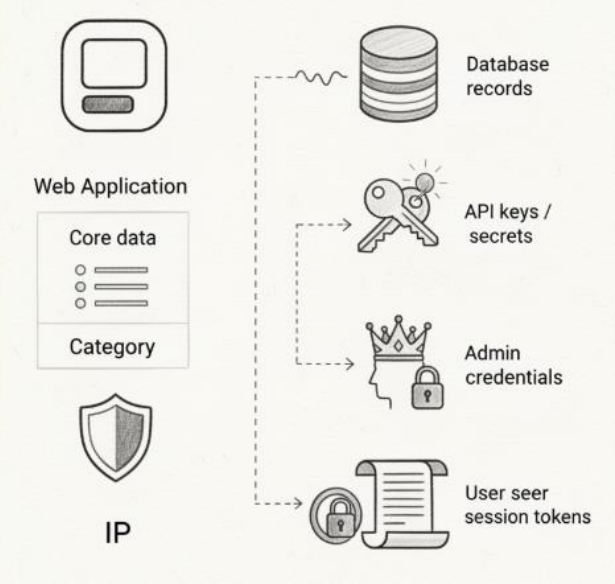
Web Application

Key assets:

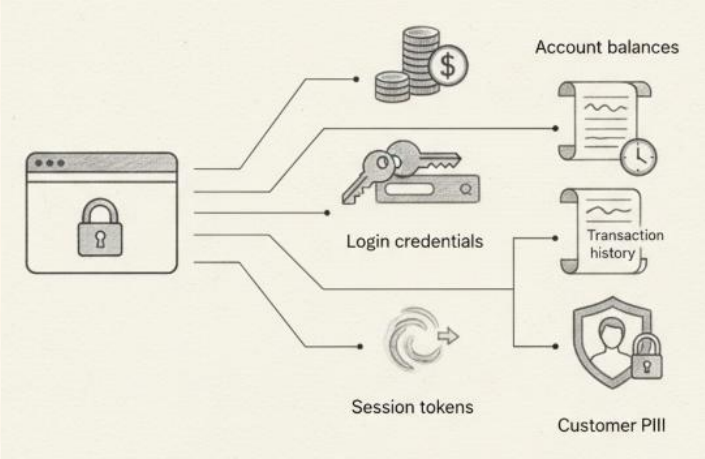
Database records	Core data
API keys / secrets	Credentials
Admin credentials	Auth
Source code	IP
User session tokens	Auth

Why it matters:

Exposed API keys allow unlimited abuse.
Database breach exposes all user data.
Admin credential theft = full compromise.



Identify Assets Example 2



Online Banking

Key assets:

Account balances	Financial
Login credentials	Auth
Transaction history	PII
Session tokens	Auth
Customer PII	Compliance

Why it matters:

A breach leaks customer funds and personal data. GDPR fines + reputational damage. Assets ranked by financial exposure.

Step 3. Think Like an Attacker

S Spoofing

Can an attacker pretend to be a legitimate user?

T Tampering

Can requests or data be modified in transit?

R Repudiation

Can a user deny having performed an action?

I Info Disclosure

Can the database be read by unauthorised users?

D Denial of Service

Can the web server be overwhelmed?

E Elevation

Can a regular user gain admin rights?

© 2026 Vlad Skorokhod

STRIDE Introduction

Letter	Threat	What breaks
S	Spoofing	Authentication
T	Tampering	Integrity
R	Repudiation	Auditability
I	Info Disclosure	Confidentiality
D	DoS	Availability
E	Privilege Escalation	Authorization

Use STRIDE on every component + data flow

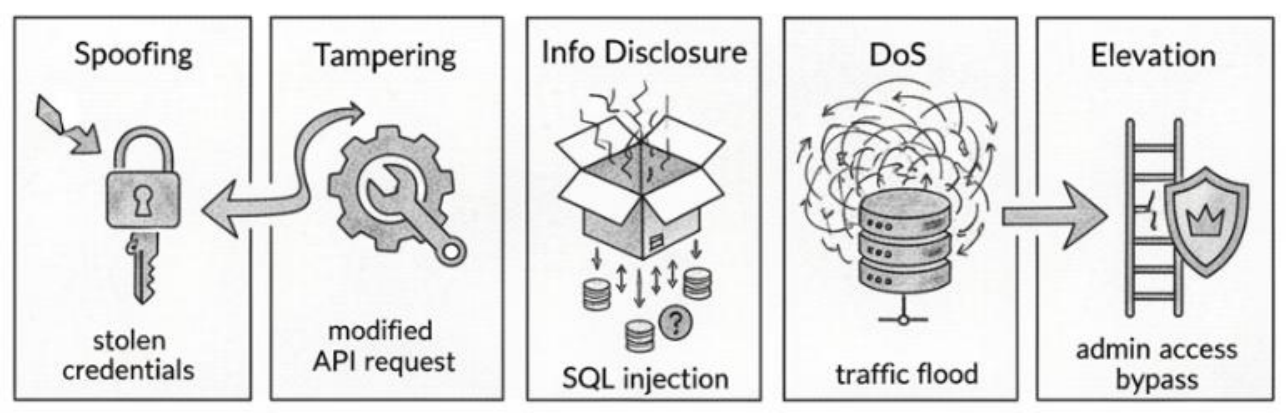
© 2026 Vlad Skorokhod

STRIDE Example (Web App)

System:

User → Web App → Database

Example threats:



STRIDE Output = Threat Model

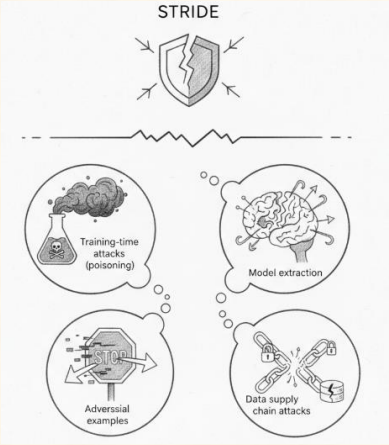
Component	Threat	Mitigation
Login	Credential theft	**MFA**
API	Request tampering	**Validation**
DB	Data leak	**Parameterized queries**
Server	DoS	**Rate limiting**

© 2026 Vlad Skorokhod

Where STRIDE Breaks for AI

STRIDE misses:

- Training-time attacks (poisoning)
- Model extraction
- Adversarial examples
- Data supply chain attacks



+ AI needs additional framework

Why Threat Modeling AI Systems Is Harder

AI systems don't fail deterministically

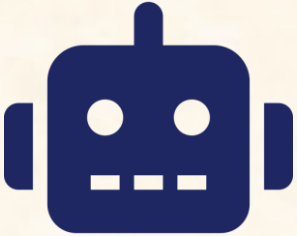
Behavior emerges from data + model + orchestration

Attackers don't just exploit code, they exploit learning systems

© 2026 Vlad Skorokhod

Enter MITRE ATLAS

AI-specific threat framework



Adds:

- Training-time attacks
- Model attacks
- Data pipeline risks

Think:

STRIDE = system view	ATLAS = AI attack view

© 2026 Vlad Skorokhod

ATLAS Structure

Start with:

Tactics = attacker goals

Techniques = how attacks work

Examples:

AML.T0020	Data poisoning
AML.T0040	Model extraction
AML.T0043	Adversarial inputs
AML.T0006	Proxy model

© 2026 Vlad Skorokhod

ATLAS Tactics: what the attacker is trying to achieve

TA0001
Reconnaissance

TA0002
Resource Development

TA0003
Initial Access

TA0004
ML Attack Staging

TA0005
Exfiltration

TA0006
Impact

https://atlas.mitre.org/tactics

Key ATLAS Techniques for Insurance AI

Four techniques that appear repeatedly in insurance ML deployments - each with a documented real-world example

AML.T0020 — **ML Attack Staging**
Poison Training Data

AML.T0040 — **Reconnaissance /Exfiltration**
ML Model Inference API

AML.T0043 — **ML Attack Staging**
Craft Adversarial Data

AML.T0006 — **Resource Development**
Create Proxy ML Model

https://atlas.mitre.org/techniques

© 2026 Vlad Skorokhod

STRIDE + ATLAS Together

Use both:

STRIDE → find **where**

ATLAS → find **how**

Example:

Component	STRIDE	ATLAS	Threat
API	Info Disclosure	`AML.T0040`	Model extraction
Training	Tampering	`AML.T0020`	Data poisoning

© 2026 Vlad Skorokhod

OWASP LLM Top 10 2025 Framework

LLM01

Prompt Injection

Attacker overrides model instructions via crafted input or injected external content

LLM02

Sensitive Info Disclosure

Model leaks PII, API keys, or memorised training data in its responses

LLM03

Supply Chain

Compromised models, datasets, or plugins introduce backdoors or malicious behaviour

LLM04

Data & Model Poisoning

Malicious training data causes the model to behave incorrectly on specific triggers

LLM05

Insecure Output Handling

LLM output used downstream without sanitisation enables XSS, SQLi or code injection

LLM06

Excessive Agency

Over-privileged agent takes unintended actions due to broad tool access or autonomy

LLM07

System Prompt Leakage

Confidential system prompt is extracted, exposing business logic or internal config

LLM08

Vector & Embedding Weakness

Attacks on RAG vector stores enable data poisoning or cross-tenant leakage

LLM09

Misinformation

Model produces plausible but false output; over-reliance leads to harmful decisions

LLM10

Unbounded Consumption

Token flooding or adversarial prompts cause excessive compute cost or DoS

CATEGORY KEY: INPUT · OUTPUT · SYSTEM · MODEL

© 2026 Vlad Skorokhod

How to break down AI systems into attack surfaces

STRIDE	MITRE ATLAS	OWASP LLM	DREAD
Structure threats by failure type	AI-specific attack techniques	GenAI failure modes	Prioritize risk by score

How to construct real attack scenarios from system architecture

Two Real Systems

1. RAG Claims Assistant (GenAI)

OpenAI API + internal knowledge base
Adjuster-facing conversational interface

2. Fraud Detection Model (ML)

Feature pipeline + Random Forest classifier
Automated claim scoring and routing

Same organization; very different risk profiles

© 2026 Vlad Skorokhod

How to Think About AI Systems

Every AI system has 3 layers:

1. Data

Training sets, retrieval docs, feature stores

2. Model

Weights, parameters, decision boundaries

3. Application / Orchestration

Prompts, APIs, pipelines, UIs

© 2026 Vlad Skorokhod

AI Attack Surface

⚠ Untrusted	⚠ Mutable	⚠ Opaque	⚠ Unverified	
User Input Prompt	**Data** Training	**Model** Inference	**Output** Response	**Action** Real-world effect
Docs	*Corpus*	*Decision*	*Text / API*	*API call / workflow*
Input can be malicious	Data can be poisoned	Model can be manipulated or compromised	Output can trigger real-world actions	

© 2026 Vlad Skorokhod

Frameworks We Will Use

STRIDE	What kind of failure?
MITRE ATLAS	How attackers exploit ML systems
OWASP LLM Top 10	GenAI-specific risks
DREAD	Which risks matter most?

No single framework is sufficient. Combine them for realistic threat models

© 2026 Vlad Skorokhod

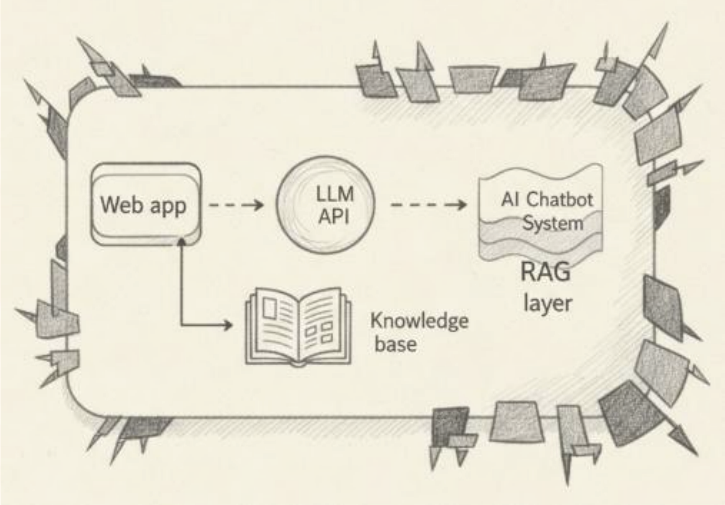
RAG Claims Assistant: Components

- User (adjuster)
- Frontend
- Backend orchestration
- Vector DB (retrieval)
- Document store
- OpenAI API
- Internal claims APIs

© 2026 Vlad Skorokhod

RAG Architecture Flow

User — **App** — **Orchestrator**



- Retriever → Vector DB
- LLM (OpenAI)
- Internal APIs

LAB 1

LAB: Explore a Simple RAG Chatbot

What Are We Protecting? (RAG)

- Claims data (PII)
- Internal policies
- Retrieved documents
- System prompt
- API credentials
- Model outputs

If you don't define assets, you can't define risk

© 2026 Vlad Skorokhod

STRIDE for RAG System Examples

Spoofing	Fake adjuster identity bypasses authorization
Tampering	Poisoned documents injected into vector DB
Repudiation	No trace or audit log of model decisions
Info Disclosure	Leaking proprietary claims data
DoS	Prompt / token flooding exhausts API quota
Elevation of Privilege	Prompt injection bypasses system prompt controls

© 2026 Vlad Skorokhod

STRIDE Is Not Enough

STRIDE tells us:

"Something can go wrong"

But NOT:

How AI systems are actually attacked in practice

→ *Use STRIDE as a starting point, then layer in AI-specific frameworks*

© 2026 Vlad Skorokhod
Proprietary & Confidential

AI-Specific Attack Mapping

Map threats to real ATLAS attack techniques:

Prompt Injection	*AML.T0051*
Data Poisoning	*AML.T0020*
Data Extraction	*AML.T0024*
Evasion	*AML.T0043*

© 2026 Vlad Skorokhod

OWASP LLM Risks & Framework Layering

OWASP LLM Top 10 (Key Risks)

LLM01	Prompt Injection
LLM02	Sensitive Data Disclosure
LLM06	Excessive Agency
LLM09	Overreliance

Framework Layering

STRIDE	→ Structure threats
ATLAS	→ Attacker techniques
OWASP	→ GenAI failure modes

Together → a realistic, layered threat model

© 2026 Vlad Skorokhod

Attack Scenario: Prompt Injection via Documents

1. Attacker uploads malicious document to the claims system
2. Document is embedded and stored in the vector DB
3. Malicious content retrieved during a normal adjuster query
4. Injected directly into the LLM prompt as context
5. Model follows attacker instructions rather than system prompt

© 2026 Vlad Skorokhod
Proprietary & Confidential

What Happens Next?

Data Exfiltration

Model reveals other adjusters' claim details

Policy Manipulation

Model outputs advice that contradicts real policy

Unauthorized API Actions

Model triggers real claim changes via internal APIs

User Deception

Adjuster receives false, attacker-controlled guidance

© 2026 Vlad Skorokhod

Why This Attack Works

Model trusts retrieved context: no separation between:

Instructions = **Data**

No trust boundary

System assumes: "retrieved content is safe" but retrieval is an attack surface

Mitigations to consider:

- Sanitize and validate retrieved documents before injection
- Separate system instructions from retrieved data in the prompt
- Limit model actions and API scope (least privilege)

© 2026 Vlad Skorokhod

DREAD Scoring

Prioritization Framework: Use this to rank and triage threats

D	R	E	A	D
Damage	**Reproducibility**	**Exploitability**	**Affected Users**	**Discoverability**
How bad if exploited?	*How easily repeated?*	*How much skill required?*	*How many impacted?*	*How easy to find?*

HIGH SCORE = HIGH PRIORITY → Prioritize remediation efforts here

Score each threat 1–5 per dimension. Aggregate to rank your mitigation backlog.

© 2026 Vlad Skorokhod

Prompt Injection Risk Score

Damage	**High**
Reproducibility	**High**
Exploitability	**High**
Affected Users	**High**
Discoverability	**High**

→ CRITICAL RISK

© 2026 Vlad Skorokhod

Sample Threat Registry: RAG Chatbot

Entry Point	Attack Technique	Attack Chain	Impacted Asset	Framework Mapping	DREAD	Priority
User Input	Prompt Injection	Input → Prompt → LLM → Output	Claims data, system prompt	STRIDE: EoP ATLAS: AML.T0051 OWASP: LLM01	9/10	High
Document Store	Data Poisoning	Upload → Embed → Retrieve → Inject → Execute	Knowledge base integrity	STRIDE: Tampering ATLAS: AML.T0020	8.5	High
Retrieval Layer	Context Injection	Retrieve → Trust → Prompt → Output	Context trust boundary	OWASP: LLM01	9	High
LLM Output	Data Exfiltration	Prompt → Model → Leak	PII, claims data	STRIDE: Info Disclosure ATLAS: AML.T0024	9	High
Internal APIs	Excessive Agency	Prompt → Tool call → Action	Claims system actions	OWASP: LLM06	8	Med
API Layer	DoS / Token Abuse	Flood → LLM overload	Availability	STRIDE: DoS	7	Med

© 2026 Vlad Skorokhod

RAG System Risk Pattern

Primary Risk	Prompt Injection
Core Weakness	Trusting retrieved context
Attack Vector	Natural language

© 2026 Vlad Skorokhod

Fraud Detection Pipeline: Components

- Claims API
- Feature engineering
- XGBoost model
- Decision threshold
- Database
- Dashboard
- Training pipeline
- Model storage

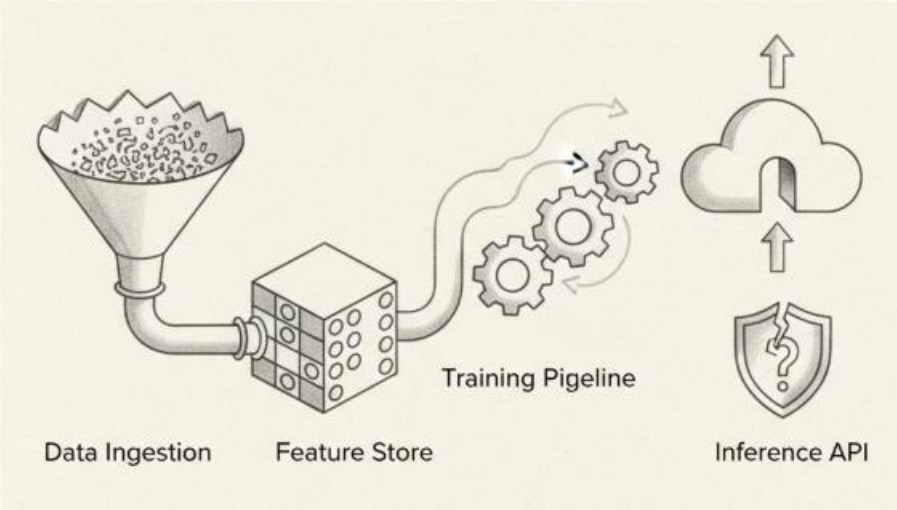
© 2026 Vlad Skorokhod

Fraud Detection Architecture Flow

Claims — Features — Model — Score — Decision

Training Pipeline

Model is retrained on incoming claims data making the training pipeline itself an attack surface



LAB 2

LAB: Fraud Detection Model

What Are We Protecting? (Fraud Model)

- 🤖 Model integrity
- 📊 Training data
- ⚙️ Feature pipeline
- 🎯 Decision thresholds
- 📈 Model predictions

© 2026 Vlad Skorokhod

STRIDE for Fraud Detection Model

Spoofing	Fake claim identity mimics legitimate patterns
Tampering	Feature manipulation to shift model score
Repudiation	No audit trail for model decision rationale
Info Disclosure	Model inversion reveals sensitive training data
DoS	Pipeline overload delays fraud scoring
Elevation	Threshold manipulation bypasses review queue

© 2026 Vlad Skorokhod

Core ML Attacks: ATLAS Mapping

Evasion · AML.T0043

Craft inputs that fool the model at inference time

Poisoning · AML.T0020

Corrupt training data to degrade or redirect model behavior

Model Extraction · AML.T0040

Query model to reconstruct a functional copy

Membership Inference · AML.T0024

Determine if a specific record was used in training

© 2026 Vlad Skorokhod

Attack Scenario: Fraud Evasion

1. Attacker submits a fraudulent insurance claim
2. Slightly modifies feature values to probe the model boundary
3. Fraud score drops just below the automated review threshold
4. Claim bypasses human review and is automatically approved

© 2026 Vlad Skorokhod
Proprietary & Confidential

Why Fraud Evasion Works

- Models rely on learned decision boundaries, not explicit rules
- Small feature perturbations can cause large prediction shifts near the boundary
- Attackers can probe the model iteratively using live predictions as feedback

System assumes: "input distribution is stable" but adversaries actively shape it

Mitigations to consider:

- Monitor score distribution over time for anomalous drift
- Rate-limit repeated claim submissions from the same source
- Require human review near the decision threshold

© 2026 Vlad Skorokhod

Fraud Evasion Risk Profile

Different risk profile than RAG: lower discoverability but sustained financial damage

© 2026 Vlad Skorokhod

Sample Threat Registry: Fraud Detection Model

Component	Threat	Attack Technique	Impact	Framework Mapping	DREAD	Priority
Input (Claims)	Identity Spoofing	Fake claims	Fraud bypass	STRIDE: Spoofing	7	Med
Feature Engineering	Data Tampering	Feature manipulation	Score distortion	STRIDE: Tampering	8	High
Model	Evasion	Adversarial input	Fraud undetected	ATLAS: AML.T0043	9	High
Training Pipeline	Poisoning	Corrupt training data	Long-term degradation	ATLAS: AML.T0020	9	High
Model API	Extraction	Query probing	IP theft	ATLAS: AML.T0040	7	Med
Output	Threshold Exploit	Score gaming	False negatives	STRIDE: EoP	8	High
Model	Membership Inference	Data leakage	Privacy breach	ATLAS: AML.T0024	7	Med

© 2026 Vlad Skorokhod

RAG vs Fraud Model Side by Side

Dimension	RAG Assistant	Fraud Model
Primary Risk	Prompt Injection	Evasion Attack
Attack Medium	Natural Language	Feature Engineering
Data Risk	Retrieval Leakage	Training Poisoning
Failure Mode	Context Trust	Decision Boundary

© 2026 Vlad Skorokhod

The Fundamental Difference

GenAI Fails Through

Language & Context

ML Fails Through

Math & Distributions

→ *Different attack surfaces require different threat models*

© 2026 Vlad Skorokhod
Proprietary & Confidential

Common Mistakes in AI Threat Modeling

Treating LLMs like APIs

LLMs have emergent behavior: prompt and context are attack surfaces

Ignoring the data layer

Training data and retrieval pipelines are high-value targets

Not modeling attack chains

Individual threats combine: model the full adversary path

Prioritizing everything equally

Use DREAD scoring to focus limited security resources

© 2026 Vlad Skorokhod

Practical Threat Modeling Workflow

1

Draw Architecture

Map all components, data flows, and trust boundaries

2

Identify Assets

What data, models, and capabilities need protecting?

3

Apply STRIDE

Ask how each component can fail adversarially

4

Map to ATLAS / OWASP

Link to known AI attack techniques

5

Build Attack Scenarios

Construct realistic end-to-end attack chains

6

Score with DREAD

Prioritize mitigations by risk level

© 2026 Vlad Skorokhod

What Matters

System-level thinking

AI threats emerge from interactions, not just individual components

Frameworks are tools

STRIDE, ATLAS, OWASP, DREAD… combine them, don't rely on one

Real risk lives in assumptions

What does the system trust that an attacker can exploit?

Prioritization is critical

Not all threats are equal: score and focus your defenses

© 2026 Vlad Skorokhod

Threat Modeling is not Algorithmic

Heuristic Overlays	Red Teaming	Analogy
• Applying "rules of thumb" to model architectures.	• Using expert-led "war gaming" to find the cracks.	• Thinking of the model as a "flawed intelligence" rather than a rigid piece of software

AI threat modeling is a heuristic and expert-driven exercise, despite the growing library of formal frameworks.

© 2026 Vlad Skorokhod

	Algorithmic **Traditional AppSec**	**Heuristic** **AI/ML Threat Modeling**
Failure Mode	Binary (Crash, Error, Leak)	Behavioral (Hallucination, Bias, Manipulation)
Testing	Unit Tests, Regression	Red Teaming, Adversarial Evaluation
Logic	Boolean / If-Then	Probabilistic / Semantic
Analogy	A locked safe with a mechanical key	A security guard who might be bribed or distracted

© 2026 Vlad Skorokhod

Ongoing Research in AI Threat Modeling

STRIDE-AI: An Approach to Identifying Vulnerabilities of Machine Learning Assets

Lara Mauri
DI – Università degli Studi di Milano
Milan, Italy
lara.mauri@unimi.it

Ernesto Damiani
C2PS – Khalifa University
Abu Dhabi, UAE
ernesto.damiani@ku.ac.ae

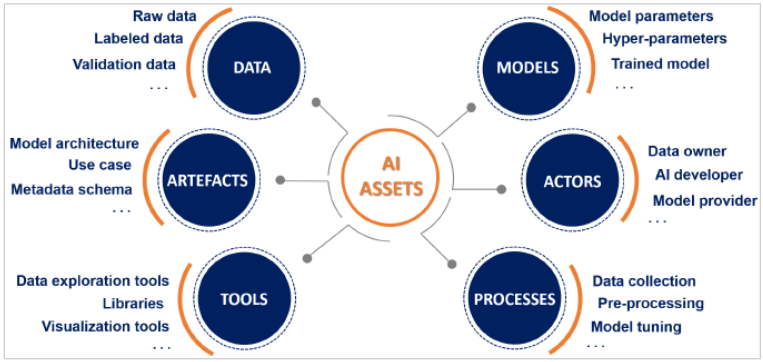


Fig. 2. Assets in the AI ecosystem.

The Problem: Traditional STRIDE focuses on software processes and data flows, fails to catch vulnerabilities of "black box" ML models and data pipelines.

Goal: Transform AI security from a "heuristic" brainstorming session into a structured, repeatable engineering process.

Source: https://github.com/LaraMauri/STRIDE-AI